

Guía Exhaustiva de Middlewares: Especificación Técnica de Backend y Reglas de Negocio

Este documento detalla exhaustivamente el funcionamiento de la capa de **Middlewares** del sistema **API Costeo**. La arquitectura de Express utiliza middlewares como interceptores en el ciclo de vida de las peticiones HTTP (`Request -> Middleware -> Controller -> Response`). Aquí se detallan sus justificaciones de negocio, flujos y especificaciones de código.

Mapa de Middlewares en el Sistema

El sistema implementa tres niveles de middlewares:

1. **Middleware de Autenticación (`auth.middleware.ts`)**: Interceptor de ruta que valida tokens de acceso contra el servidor de identidad corporativa.
 2. **Middlewares de Aplicación Global (`src/app.ts`)**: Configurados a nivel de servidor Express para dar soporte a parseo de bodies gigantes, seguridad CORS, serving de archivos estáticos y control de excepciones.
 3. **Middleware de Subida de Archivos (Multer)**: Configurado a nivel de ruteadores específicos para interceptar archivos PDF e imágenes en la memoria RAM y prepararlos para su envío a Cloudinary y APIs de OCR.
-

1. Middleware de Autenticación (`auth.middleware.ts`)

A. Lógica y Procesos de Negocio

- **Protección de Datos Sensibles**: Las estimaciones financieras, reportes de costo y asignaciones de presupuesto son de carácter confidencial. El middleware garantiza que ningún usuario anónimo pueda consultar o modificar las bases de datos.
- **Trazabilidad de Auditoría (Identity Injection)**: Al validar al usuario, el middleware extrae su identidad e inyecta la metadata del usuario en la solicitud (`req.user`). Esto

permite que los controladores que realizan operaciones de base de datos (como creación de reportes o commits de trazabilidad) guarden automáticamente las firmas del creador/editor (`userId` , `userName` , `emailUser`), asegurando la trazabilidad de cada acción.

- **Regla del Mock de Desarrollo:** Para evitar bloqueos operativos cuando el microservicio central de autenticación OAuth no esté disponible o el programador esté trabajando sin conexión de red:
 - Si la variable de entorno `NODE_ENV` está en `'development'` y el token de autorización coincide exactamente con el valor `'dev-mock-token-xyz-123'` , el middleware simula la aprobación de un usuario administrador ficticio ("Usuario Desarrollo", con correo `dev@cicsa.com` y rol `admin`), cediendo el paso de inmediato.

B. Especificación del Backend y Flujo Técnico

- **Archivo físico:** [auth.middleware.ts](#)
- **Firma de la función:** `export const authMiddleware = async (req: Request, res: Response, next: NextFunction)`

Extensión de Tipos de Express (TypeScript)

Para evitar errores de compilación de TypeScript al asignar `req.user` , se implementa un archivo de definición global en [express.d.ts](#):

```
import { User } from "../utils/typesInterfaces";

declare global {
  namespace Express {
    interface Request {
      user: User;
    }
  }
}
```

La interfaz `User` se define en [typesInterfaces.ts](#) con la siguiente estructura:

```
export interface User {  
  id: string;  
  name: string;  
  email: string;  
  image?: string;  
  role: string;  
}
```

Análisis de Código Paso a Paso:

1. Detección de Entorno:

```
const isDev = process.env.NODE_ENV === 'development';
```

2. Extracción de la Cabecera de Autorización:

```
const authHeader = req.headers.authorization;
```

El middleware inspecciona los encabezados HTTP. Si `authHeader` está ausente o no comienza con el prefijo `Bearer`, se deniega el acceso retornando un código **HTTP 401 (Unauthorized)** con el JSON `{ message: 'No autorizado - Token faltante' }`.

3. Aislamiento del Token:

```
const token = authHeader.split(' ')[1];
```

Divide la cabecera por espacios y extrae el token en la posición 1.

4. Bypass de Desarrollo (Mocking):

Si la condición `isDev` es verdadera y el `token` es idéntico a `'dev-mock-token-xyz-123'`, inyecta el objeto mock en la petición:

```
req.user = {  
  id: 'mock-id-123',  
  name: 'Usuario Desarrollo',  
  email: 'dev@cicsa.com',  
  role: 'admin'  
};  
return next();
```

5. Validación Remota contra OAuth:

Si no es un bypass, el middleware realiza una petición asíncrona mediante `fetch` al servidor de identidad externo:

```
const response = await
fetch(`${process.env.AUTH_SERVICE_URL}/api/auth/oauth2/userinfo`, {
  headers: {
    Authorization: `Bearer ${token}`,
  },
});
```

6. Manejo de Expiración e Inconsistencia:

- Si la respuesta de la API de identidad no es exitosa (`!response.ok`), devuelve un **HTTP 401 (Unauthorized)** `{ message: "Sesión expirada" }`.
- Si ocurre un error de red o de conexión (bloqueado en el bloque `catch`), se intercepta la excepción devolviendo un **HTTP 401** `{ message: "No autenticado" }`.

7. Inyección y Continuidad:

Si la API OAuth responde de manera exitosa, el middleware mapea `req.user = userData.user` y llama a `next()` para transferir la solicitud al siguiente middleware o controlador de la ruta.

2. Middlewares de Aplicación Global (`src/app.ts`)

Configurados y cargados en la instancia de la aplicación en [app.ts](#).

A. Lógica y Procesos de Negocio

- **Soporte para Ingesta Contable Masiva:** Al importar datos de trazabilidad mensual o subir listados de presupuestos (TEBs), el volumen de información en el cuerpo de la petición puede ser de varios megabytes. El middleware de JSON se amplía para evitar rechazos por límite de tamaño de carga (*Payload Too Large*).
- **Seguridad CORS:** Impide que sitios web maliciosos no autorizados invoquen los endpoints de la API de costeo. En producción, restringe los orígenes permitidos a la URL autorizada por la empresa.
- **Visualización de Evidencia del OCR:** Cuando el sistema digitaliza bitácoras, divide los archivos PDF y genera imágenes temporales. El serving estático permite mapear temporalmente estas imágenes para que el capturista valide en pantalla que los datos que el OCR de la IA interpretó coinciden perfectamente con el documento físico original antes de persistir la información.

- **Sanitización de Errores Críticos:** Atrapa cualquier excepción no controlada en el backend (errores de base de datos MongoDB, fallos en APIs de OCR) y responde con un formato estructurado de error, ocultando del cliente detalles técnicos del servidor.

B. Especificación del Backend y Flujo Técnico

- **Morgan Logger (`app.use(morgan('dev'))`):** Imprime en consola un resumen de cada petición HTTP entrante (método, ruta, código de estado HTTP y tiempo de respuesta en milisegundos), facilitando la depuración durante el desarrollo local.
- **JSON Parser con Límite Ampliado (`app.use(express.json({ limit: '100mb' }))`):**
 - Intercepta solicitudes con cabecera `Content-Type: application/json`.
 - Se configura el parámetro `{ limit: '100mb' }` para tolerar cuerpos de hasta 100 Megabytes, indispensable para la ingesta de arrays masivos de trazabilidad temporal.
- **URL Encoded Parser (`app.use(express.urlencoded({ extended: true }))`):**
 - Parsea solicitudes codificadas en URL (formularios convencionales).
 - La opción `{ extended: true }` permite el parseo de estructuras de datos anidadas y complejas mediante la biblioteca `qs` interna de Express.
- **Intercepción CORS (`app.use(cors(...))`):**
 - Determina dinámicamente la regla de origen:


```
const ALLOWED_ORIGIN = process.env.NODE_ENV === 'development' ? true : process.env.ALLOWED_ORIGIN;
```
 - Habilita los métodos HTTP: `GET, POST, PUT, DELETE, PATCH, OPTIONS`.
 - Permite cabeceras de autorización y desarrollo: `Content-Type, Authorization, X-Is-Development, Accept`.
 - Permite el intercambio de cookies y cabeceras de autenticación fijando `credentials: true`.
- **Servidor de Contenido Estático (`app.use('/static', express.static(...))`):**
 - Mapea la ruta URL pública `/static` al directorio físico `scanned_images` en la raíz del servidor de Node.
 - Sirve las imágenes segmentadas por página de los PDFs procesados por el OCR para su visualización en el frontend.
- **Manejador de Errores Global:**

- Ubicado al final de la cadena de enrutamiento. Captura llamadas a `next(err)` de cualquier controlador:

```
app.use((err: any, req: Request, res: Response, next: NextFunction)
=> {
  res.status(500).json({
    error: true,
    message: err.message,
  });
});
```

- Devuelve un código **HTTP 500 (Internal Server Error)** y responde con una estructura limpia de error JSON.

3. Middleware de Recepción de Archivos (Multer)

A. Lógica y Procesos de Negocio

- **Seguridad de Datos y Almacenamiento en Memoria RAM:** El sistema procesa archivos PDFs confidenciales de TEBs y bitácoras del cliente. Para mitigar riesgos de almacenamiento físico y accesos no autorizados en el disco duro del servidor de Express, el middleware almacena los archivos recibidos directamente en la memoria RAM del servidor de forma temporal.
- **Persistencia Delegada:** La persistencia de los archivos PDF a largo plazo se delega de manera directa a la nube segura de Cloudinary, evitando copias locales del documento.

B. Especificación del Backend y Flujo Técnico

- **Ubicación:** Se importa e instancia en los enrutadores de rutas de archivos (como `BaseTeb.route.ts`, `DailyReportScanner.route.ts` y `fictitiousTeb.route.ts`).
- **Instancia:**

```
import multer from "multer";
const storage = multer.memoryStorage();
const upload = multer({ storage: storage });
```

Al utilizar `multer.memoryStorage()` , los archivos cargados se configuran como objetos que contienen el buffer del archivo (`file.buffer`), tipo MIME (`file.mimetype`) y nombre original (`file.originalname`).

- **Implementación en Rutas:**

- Se inyecta en las rutas de carga mediante `upload.array('files')` .
- Parsea el cuerpo multipart de la petición y deposita el arreglo de archivos en la propiedad `req.files` del objeto de solicitud.
- Posteriormente, el controlador accede a `req.files` as `Express.Multer.File[]` y lee directamente el buffer del archivo en memoria para enviarlo a la API de Azure o subirlo al repositorio de almacenamiento de Cloudinary.